

utils.py (Q&A Documentation)

Q: What does this file provide? **A:** This file provides shared metrics (MAE, XAUC, KL) and TPM-specific encoding/decoding helpers for tree-based bucket prediction.

Q: Why does the repo use XAUC for a regression-like task? **A:** XAUC measures whether larger predictions correspond to larger labels (a ranking-consistency signal). It complements MAE (absolute error).

Q: What does this file export (public API)? **A:** The public API is the set of classes/functions other modules import (sometimes re-exported by package initializers). The key top-level definitions detected here are:

```
def get_playtime_percentiles_range(...) # function
def get_tree_classify_loss(...) # function
def get_tree_encoded_label(...) # function
def get_tree_encoded_value(...) # function
class InversePairsCalc # methods: InversePairs
def eval_xauc(...) # function
def eval_auc(...) # function
def eval_mae(...) # function
def eval_kl(...) # function
```

Q: What are the main dependencies (imports) and why do they matter? **A:** Imports show whether this file is model code (torch), data code (pandas), or evaluation/analysis (sklearn). They also determine runtime requirements.

```
import math
import numpy as np
import matplotlib.pyplot as plt
import os
import math
import torch
import torch.nn.functional as F
from sklearn.metrics import roc_auc_score

import torch
import numpy as np

def get_playtime_percentiles_range(dataloader, wr_bucknum, _device):
    all_play_time = []
    for _, (_, label) in enumerate(dataloader):
        play_time = label
        all_play_time.append(play_time)
```

```

all_play_time = torch.cat(all_play_time, dim=0)
play_time_np = all_play_time.cpu().numpy()
percen_value = np.percentile(play_time_np, np.linspace(0.0, 100.0, num=wr_bucknum + 1).astype(np.float32)).tolist()
bucket_begins = torch.tensor(percen_value[:-1], dtype=torch.float32, device=_device).unsqueeze(0)
bucket_ends = torch.tensor(percen_value[1:], dtype=torch.float32, device=_device).unsqueeze(0)
return bucket_begins, bucket_ends

def get_tree_classify_loss(label_dict, weight_dict, label_encoding_predict, tree_num_intervals=32):
    auxiliary_loss_ = 0.0
    height = int(math.log2(tree_num_intervals))
    for i in range(height):
        for j in range(2**i):
            interval_label = label_dict[1000*i + j].reshape(-1, 1)
            interval_weight = weight_dict[1000*i + j].reshape(-1, 1)
            interval_preds = label_encoding_predict[:, 2**i - 1 + j].view(-1,1)
            interval_loss = F.binary_cross_entropy_with_logits(interval_preds, interval_label, weight=interval_weight)
            auxiliary_loss_ += interval_loss
    final_loss = auxiliary_loss_ / (tree_num_intervals - 1.0)
    return final_loss.float()

def get_tree_encoded_label(label, tree_num_intervals, begins, ends, name="label_encoding"):
    label_dict = {}
    weight_dict = {}
    height = int(math.log2(tree_num_intervals))
    for i in range(height):
        for j in range(2**i):
            temp_ind = max(int(tree_num_intervals * 1.0 / (2**i) * j) - 1, 0)

            if j == 0:
                weight_temp = torch.where(label < begins[:, temp_ind].reshape(-1, 1), torch.zeros_like(label), torch.ones_like(label))
            else:
                weight_temp = torch.where(label < ends[:, temp_ind].reshape(-1, 1), torch.zeros_like(label), torch.ones_like(label))

            label_dict[1000*i + j] = label[:, temp_ind].reshape(-1, 1)
            weight_dict[1000*i + j] = weight_temp[:, temp_ind].reshape(-1, 1)
    return label_dict, weight_dict

```

Q: What is `get_playtime_percentiles_range` and what responsibility does it have? **A:** This function is a core unit in the pipeline. Its responsibility is inferred from its name and how run scripts call it.

Q: What does the implementation look like for `get_playtime_percentiles_range`? **A:** Excerpt from the file:

```

def get_playtime_percentiles_range(dataloader, wr_bucknum, _device):
    all_play_time = []
    for _, (_, label) in enumerate(dataloader):
        play_time = label
        all_play_time.append(play_time)
    all_play_time = torch.cat(all_play_time, dim=0)
    play_time_np = all_play_time.cpu().numpy()
    percen_value = np.percentile(play_time_np, np.linspace(0.0, 100.0, num=wr_bucknum + 1).astype(np.float32)).tolist()
    bucket_begins = torch.tensor(percen_value[:-1], dtype=torch.float32, device=_device).unsqueeze(0)
    bucket_ends = torch.tensor(percen_value[1:], dtype=torch.float32, device=_device).unsqueeze(0)
    return bucket_begins, bucket_ends

```

Q: What is `get_tree_classify_loss` and what responsibility does it have? **A:** This function is a core unit in the pipeline. Its responsibility is inferred from its name and how run

scripts call it.

Q: What does the implementation look like for `get_tree_classify_loss`? **A:** Excerpt from the file:

```
def get_tree_classify_loss(label_dict, weight_dict, label_encoding_predict, tree_num_intervals=32):
    auxiliary_loss_ = 0.0
    height = int(math.log2(tree_num_intervals))
    for i in range(height):
        for j in range(2**i):
            interval_label = label_dict[1000*i + j].reshape(-1, 1)
            interval_weight = weight_dict[1000*i + j].reshape(-1, 1)
            interval_preds = label_encoding_predict[:, 2**i - 1 + j].view(-1,1)
            interval_loss = F.binary_cross_entropy_with_logits(interval_preds, interval_label, weight=interval_weight)
            auxiliary_loss_ += interval_loss
    final_loss = auxiliary_loss_ / (tree_num_intervals - 1.0)
    return final_loss.float()
```

Q: What is `get_tree_encoded_label` and what responsibility does it have? **A:** This function is a core unit in the pipeline. Its responsibility is inferred from its name and how run scripts call it.

Q: What does the implementation look like for `get_tree_encoded_label`? **A:** Excerpt from the file:

```
def get_tree_encoded_label(label, tree_num_intervals, begins, ends, name="label_encoding"):
    label_dict = {}
    weight_dict = {}
    height = int(math.log2(tree_num_intervals))
    for i in range(height):
        for j in range(2**i):
            temp_ind = max(int(tree_num_intervals * 1.0 / (2**i) * j) - 1, 0)

            if j == 0:
                weight_temp = torch.where(label < begins[:, temp_ind].reshape(-1, 1), torch.zeros_like(label), torch.ones_like(label))
            else:
                weight_temp = torch.where(label < ends[:, temp_ind].reshape(-1, 1), torch.zeros_like(label), torch.ones_like(label))

            temp_ind = max(int(tree_num_intervals * 1.0 / (2**i) * (j + 1)) - 1, 0)
            weight_temp = torch.where(label < ends[:, temp_ind].reshape(-1, 1), weight_temp, torch.zeros_like(label))

            temp_ind = max(int(tree_num_intervals * (1.0 / (2**i) * j + 1.0 / (2**(i + 1)))) - 1, 0)
            label_temp = torch.where(label < ends[:, temp_ind].reshape(-1, 1), torch.zeros_like(label), torch.ones_like(label))

            label_dict[1000 * i + j] = label_temp
            weight_dict[1000 * i + j] = weight_temp

    return label_dict, weight_dict
```

Q: What is `get_tree_encoded_value` and what responsibility does it have? **A:** This function is a core unit in the pipeline. Its responsibility is inferred from its name and how run scripts call it.

Q: What does the implementation look like for `get_tree_encoded_value`? **A:** Excerpt from the file:

```
def get_tree_encoded_value(label_encoding_predict, tree_num_intervals, begins, ends, name="encoded_playtime"):
    height = int(math.log2(tree_num_intervals))
```

```

encoded_prob_list = []

temp_encoded_playtime = (begins + ends) / 2.0
encoded_playtime = temp_encoded_playtime

batch_size = label_encoding_predict.size(0)
device = label_encoding_predict.device

for i in range(tree_num_intervals):
    temp = torch.zeros(batch_size, dtype=torch.float32, device=device)
    cur_code = 2 ** height - 1 + i

    for j in range(1, height + 1):
        classifier_branch = cur_code % 2
        classifier_idx = (cur_code - 1) // 2

        probs = label_encoding_predict[:, classifier_idx]
        condition = torch.tensor(classifier_branch == 1, dtype=torch.bool, device=device)
        log_p = torch.where(condition, torch.log(1.0 - probs + 0.00001), torch.log(probs + 0.00001))
        temp += log_p

        cur_code = classifier_idx
    encoded_prob_list.append(temp)
encoded_prob = torch.exp(torch.stack(encoded_prob_list, dim=1))
encoded_playtime = torch.sum(temp_encoded_playtime * encoded_prob, dim=-1, keepdim=True)

e_x2 = torch.sum((encoded_playtime ** 2) * encoded_prob, dim=-1, keepdim=True)
square_of_e_x = encoded_playtime ** 2
var = torch.sqrt(torch.abs(e_x2 - square_of_e_x) + 1e-8)

return encoded_playtime.float(), torch.sum(var).float()

```

Q: What is InversePairsCalc and what responsibility does it have? **A:** This class is a core unit in the pipeline. Its responsibility is inferred from its name and how run scripts call it.

Q: What are the important methods on InversePairsCalc? **A:** In this repo, methods like forward/loss/predict define computation and training targets. The methods found are:

InversePairs

Q: What does the implementation look like for InversePairsCalc? **A:** Excerpt from the file:

```

class InversePairsCalc:
    def InversePairs(self, data):
        if not data :
            return False
        if len(data)==1 :
            return 0
        def merge(tuple_fir,tuple_sec):
            array_before = tuple_fir[0]
            cnt_before = tuple_fir[1]
            array_after = tuple_sec[0]
            cnt_after = tuple_sec[1]
            cnt = cnt_before+cnt_after
            flag = len(array_after)-1
            array_merge = []
            for i in range(len(array_before)-1,-1,-1):
                while array_before[i]<=array_after[flag] and flag>=0 :
                    array_merge.append(array_after[flag])
                    flag -= 1
                if flag == -1 :
                    break

```

```

        else:
            array_merge.append(array_before[i])
            cnt += (flag+1)
    if flag == -1 :
        for j in range(i,-1,-1):
            array_merge.append(array_before[j])
    else:
        for j in range(flag ,-1,-1):
            array_merge.append(array_after[j])
    return array_merge[::-1],cnt

def mergesort(array):
    if len(array)==1:
        return (array,0)
    cut = math.floor(len(array)/2)
    tuple_fir=mergesort(array[:cut])
    tuple_sec=mergesort(array[cut:])
    return merge(tuple_fir, tuple_sec)
return mergesort(data)[1]

```

Q: What is eval_xauc and what responsibility does it have? **A:** This function is a core unit in the pipeline. Its responsibility is inferred from its name and how run scripts call it.

Q: What does the implementation look like for eval_xauc? **A:** Excerpt from the file:

```

def eval_xauc(labels, pres):
    label_preds = zip(labels.reshape(-1), pres.reshape(-1))
    sorted_label_preds = sorted(
        label_preds, key=lambda lc: lc[1], reverse=True)
    label_preds_len = len(sorted_label_preds)
    pairs_cnt = label_preds_len * (label_preds_len-1) / 2

    labels_sort = [ele[0] for ele in sorted_label_preds]
    S=InversePairsCalc()
    total_positive = S.InversePairs(labels_sort)
    xauc = total_positive / pairs_cnt
    return xauc

```

Q: What is eval_auc and what responsibility does it have? **A:** This function is a core unit in the pipeline. Its responsibility is inferred from its name and how run scripts call it.

Q: What does the implementation look like for eval_auc? **A:** Excerpt from the file:

```

def eval_auc(labels, pres):
    auc = roc_auc_score(labels, pres)
    return auc

```

Q: What is eval_mae and what responsibility does it have? **A:** This function is a core unit in the pipeline. Its responsibility is inferred from its name and how run scripts call it.

Q: What does the implementation look like for eval_mae? **A:** Excerpt from the file:

```

def eval_mae(labels, scores):
    return np.mean(np.abs(labels - scores))

```

Q: What is eval_kl and what responsibility does it have? **A:** This function is a core unit in the pipeline. Its responsibility is inferred from its name and how run scripts call it.

Q: What does the implementation look like for `eval_kl`? **A:** Excerpt from the file:

```
def eval_kl(samples_p, samples_q, bins=100, epsilon=1e-10):
    # 计算直方图分箱概率
    hist_p, bin_edges = np.histogram(samples_p, bins=bins, density=True)
    hist_q, _ = np.histogram(samples_q, bins=bin_edges, density=True)

    # 计算每个分箱的宽度 (用于归一化)
    bin_width = np.diff(bin_edges)
    hist_p = hist_p * bin_width # 转为概率质量
    hist_q = hist_q * bin_width

    # 防止零概率
    hist_p = np.clip(hist_p, epsilon, None)
    hist_q = np.clip(hist_q, epsilon, None)

    # 计算KL散度
    kl = np.sum(hist_p * np.log(hist_p / hist_q))
    return kl
```

Q: Example: end-to-end data flow involving this file **A:** Core pipeline shape (schematic):

```
mae = eval_mae(labels, preds)
xauc = eval_xauc(labels, preds)
kl = eval_kl(labels, preds)
```

Q: Full source code of the Python file **A:** The complete file is included verbatim for reference.

```
import math
import numpy as np
import matplotlib.pyplot as plt
import os
import math
import torch
import torch.nn.functional as F
from sklearn.metrics import roc_auc_score

import torch
import numpy as np

def get_playtime_percentiles_range(dataloader, wr_bucknum, _device):
    all_play_time = []
    for _, (_, label) in enumerate(dataloader):
        play_time = label
        all_play_time.append(play_time)
    all_play_time = torch.cat(all_play_time, dim=0)
    play_time_np = all_play_time.cpu().numpy()
    percen_value = np.percentile(play_time_np, np.linspace(0.0, 100.0, num=wr_bucknum + 1).astype(np.float32)).tolist()
    bucket_begins = torch.tensor(percen_value[:-1], dtype=torch.float32, device=_device).unsqueeze(0)
    bucket_ends = torch.tensor(percen_value[1:], dtype=torch.float32, device=_device).unsqueeze(0)
    return bucket_begins, bucket_ends

def get_tree_classify_loss(label_dict, weight_dict, label_encoding_predict, tree_num_intervals=32):
    auxiliary_loss_ = 0.0
    height = int(math.log2(tree_num_intervals))
    for i in range(height):
        for j in range(2**i):
            interval_label = label_dict[1000*i + j].reshape(-1, 1)
            interval_weight = weight_dict[1000*i + j].reshape(-1, 1)
            interval_preds = label_encoding_predict[:, 2**i - 1 + j].view(-1,1)
            interval_loss = F.binary_cross_entropy_with_logits(interval_preds, interval_label, weight=interval_weight)
            auxiliary_loss_ += interval_loss
    final_loss = auxiliary_loss_ / (tree_num_intervals - 1.0)
```

```

return final_loss.float()

def get_tree_encoded_label(label, tree_num_intervals, begins, ends, name="label_encoding"):
    label_dict = {}
    weight_dict = {}
    height = int(math.log2(tree_num_intervals))
    for i in range(height):
        for j in range(2**i):
            temp_ind = max(int(tree_num_intervals * 1.0 / (2**i) * j) - 1, 0)

            if j == 0:
                weight_temp = torch.where(label < begins[:, temp_ind].reshape(-1, 1), torch.zeros_like(label), torch.ones_like(label))
            else:
                weight_temp = torch.where(label < ends[:, temp_ind].reshape(-1, 1), torch.zeros_like(label), torch.ones_like(label))

            temp_ind = max(int(tree_num_intervals * 1.0 / (2**i) * (j + 1)) - 1, 0)
            weight_temp = torch.where(label < ends[:, temp_ind].reshape(-1, 1), weight_temp, torch.zeros_like(label))

            temp_ind = max(int(tree_num_intervals * (1.0 / (2**i) * j + 1.0 / (2**(i + 1)))) - 1, 0)
            label_temp = torch.where(label < ends[:, temp_ind].reshape(-1, 1), torch.zeros_like(label), torch.ones_like(label))

            label_dict[1000 * i + j] = label_temp
            weight_dict[1000 * i + j] = weight_temp

    return label_dict, weight_dict

def get_tree_encoded_value(label_encoding_predict, tree_num_intervals, begins, ends, name="encoded_playtime"):
    height = int(math.log2(tree_num_intervals))
    encoded_prob_list = []

    temp_encoded_playtime = (begins + ends) / 2.0
    encoded_playtime = temp_encoded_playtime

    batch_size = label_encoding_predict.size(0)
    device = label_encoding_predict.device

    for i in range(tree_num_intervals):
        temp = torch.zeros(batch_size, dtype=torch.float32, device=device)
        cur_code = 2 ** height - 1 + i

        for j in range(1, height + 1):
            classifier_branch = cur_code % 2
            classifier_idx = (cur_code - 1) // 2

            probs = label_encoding_predict[:, classifier_idx]
            condition = torch.tensor(classifier_branch == 1, dtype=torch.bool, device=device)
            log_p = torch.where(condition, torch.log(1.0 - probs + 0.00001), torch.log(probs + 0.00001))
            temp += log_p

            cur_code = classifier_idx
            encoded_prob_list.append(temp)
        encoded_prob = torch.exp(torch.stack(encoded_prob_list, dim=1))
        encoded_playtime = torch.sum(temp_encoded_playtime * encoded_prob, dim=-1, keepdim=True)

    e_x2 = torch.sum((encoded_playtime ** 2) * encoded_prob, dim=-1, keepdim=True)
    square_of_e_x = encoded_playtime ** 2
    var = torch.sqrt(torch.abs(e_x2 - square_of_e_x) + 1e-8)

    return encoded_playtime.float(), torch.sum(var).float()

class InversePairsCalc:
    def InversePairs(self, data):
        if not data:
            return False
        if len(data)==1:
            return 0

```

```

def merge(tuple_fir,tuple_sec):
    array_before = tuple_fir[0]
    cnt_before = tuple_fir[1]
    array_after = tuple_sec[0]
    cnt_after = tuple_sec[1]
    cnt = cnt_before+cnt_after
    flag = len(array_after)-1
    array_merge = []
    for i in range(len(array_before)-1,-1,-1):
        while array_before[i]<=array_after[flag] and flag>=0 :
            array_merge.append(array_after[flag])
            flag -= 1
        if flag == -1 :
            break
        else:
            array_merge.append(array_before[i])
            cnt += (flag+1)
    if flag == -1 :
        for j in range(i,-1,-1):
            array_merge.append(array_before[j])
    else:
        for j in range(flag ,-1,-1):
            array_merge.append(array_after[j])
    return array_merge[::-1],cnt

def mergesort(array):
    if len(array)==1:
        return (array,0)
    cut = math.floor(len(array)/2)
    tuple_fir=mergesort(array[:cut])
    tuple_sec=mergesort(array[cut:])
    return merge(tuple_fir, tuple_sec)
return mergesort(data)[1]

def eval_xauc(labels, pres):
    label_preds = zip(labels.reshape(-1), pres.reshape(-1))
    sorted_label_preds = sorted(
        label_preds, key=lambda lc: lc[1], reverse=True)
    label_preds_len = len(sorted_label_preds)
    pairs_cnt = label_preds_len * (label_preds_len-1) / 2

    labels_sort = [ele[0] for ele in sorted_label_preds]
    S=InversePairsCalc()
    total_positive = S.InversePairs(labels_sort)
    xauc = total_positive / pairs_cnt
    return xauc

def eval_auc(labels, pres):
    auc = roc_auc_score(labels, pres)
    return auc

def eval_mae(labels, scores):
    return np.mean(np.abs(labels - scores))

def eval_kl(samples_p, samples_q, bins=100, epsilon=1e-10):
    # 计算直方图分箱概率
    hist_p, bin_edges = np.histogram(samples_p, bins=bins, density=True)
    hist_q, _ = np.histogram(samples_q, bins=bin_edges, density=True)

    # 计算每个分箱的宽度 (用于归一化)
    bin_width = np.diff(bin_edges)
    hist_p = hist_p * bin_width # 转为概率质量
    hist_q = hist_q * bin_width

    # 防止零概率
    hist_p = np.clip(hist_p, epsilon, None)
    hist_q = np.clip(hist_q, epsilon, None)

```



```
# 计算KL散度
kl = np.sum(hist_p * np.log(hist_p / hist_q))
return kl
```